

HiDiTingV100 2131E VoLTE

开发指南

文档版本 00B03
发布日期 2025-07-30

前言

概述

本文档主要介绍通过 Brandy 和 Hi2131E 搭配，实现 VoLTE 功能的开发相关内容，包括 SDK 软硬件架构、驱动&接口、系统管理、协议软件和应用软件等，简要描述了 VoLTE 各接口、功能模块的功能和工作原理，通过实例按场景介绍各接口和功能模块的开发过程及注意事项。

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。

符号	说明
 须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B03	2025-07-30	- 更新 “2.1.1 合入 VoLTE 功能修改” 。 - 更新 “3.3 VoLTE 相关 NV” 。
00B02	2025-06-27	更新 “2.2 DiTing 单板外接连线说明” 。
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 概述.....	1
1.1 VoLTE 架构说明.....	1
1.2 开发规范和约束.....	3
2 环境搭建.....	4
2.1 编译构建.....	4
2.1.1 合入 VoLTE 功能修改.....	4
2.1.2 Brandy 侧版本编译.....	5
2.1.3 Hi2131E 侧版本编译.....	5
2.2 DiTing 单板外接连线说明.....	5
2.3 VoLTE 拨号通话流程.....	6
2.3.1 通过 AT 命令拨打电话.....	6
2.3.2 通过 RIL 接口拨打电话.....	8
3 对外接口描述	10
3.1 RIL 相关接口	11
3.1.1 设备信息接口	11
3.1.1.1 获取软件版本号.....	11
3.1.1.2 获取 IMEI 设备识别号	12
3.1.1.3 设定 VoLTE 维测日志级别.....	12
3.1.2 SIM 卡信息接口	13
3.1.2.1 判断 Hi2131E 的 SIM 卡是否在位	13
3.1.2.2 判断 Hi2131E 的 SIM 卡 PIN 码是否被锁.....	14
3.1.2.3 判断 Hi2131E 的 SIM 卡 PUK 码是否被锁	14

3.1.2.4 获取 Hi2131E 的 SIM 卡 IMSI 号	15
3.1.2.5 获取 Hi2131E 的 SIM 卡 msisdn	15
3.1.3 驻网状态接口	16
3.1.3.1 获取驻网状态	16
3.1.3.2 显示运营商信息	17
3.1.3.3 获取 IMS 驻网状态	17
3.1.3.4 是否 PDP 驻网, 显示 IP 地址	18
3.1.3.5 手动搜网, 并显示可选网络	18
3.1.4 通话接口	19
3.1.4.1 设置自动接听	19
3.1.4.2 拨打电话	20
3.1.4.3 手动接听电话	20
3.1.4.4 挂断电话	21
3.1.5 AT 指令通道	22
3.1.5.1 AT 指令发送	22
3.1.5.2 同步 AT 命令发送	22
3.2 VoLTE 相关 AT 命令	23
3.2.1 发起呼叫请求: D	23
3.2.2 接听来电: A	24
3.2.3 挂断电话: H	24
3.2.4 设置自动接听: S0	25
3.2.5 查询当前呼叫信息: +CLCC	26
3.2.6 查询 IMS 域注册状态: +CIREG	27
3.2.7 主动上报 IMS 域注册状态: +CIREGU	28
3.2.8 查询 IMS 域的虚拟号码: +VOLTEIMPU	29
3.2.9 IMS 服务状态更新指示: +IMSSRVSTATUS	30
3.2.10 来电指示: IRING	31
3.2.11 发起语音呼叫指示: +ORIG	31
3.2.12 UE 收到网络指示: +CONF	32
3.2.13 呼叫接通指示: +CONN	32
3.2.14 通话结束指示: +CEND	33

3.2.15 呼叫状态上报: +CALLSTATE 35

3.2.16 主动上报语音带宽信息: +CHANNELINFO 36

3.2.17 上报 IMS 注册失败原因: +IMSREGERR 37

3.2.18 IMS 域被叫处理状态信息主动上报: +IMSMTRPT 38

3.2.19 关闭 IMS 功能开关: +IMSSWITCH 40

3.3 VoLTE 相关 NV 40

插图目录

图 1-1 VoLTE 系统架构图 2

表格目录

表 1-1 VoLTE 系统模块描述..... 2

表 1-2 IMS 子系统模块划分 2

表 2-1 DiTing 单板外接连线说明..... 5

表 3-1 RIL 接口实现方式..... 10

表 3-2 参数说明11

表 3-3 返回值说明11

表 3-4 参数说明 12

表 3-5 返回值说明 12

表 3-6 参数说明 12

表 3-7 参数说明 13

表 3-8 返回值说明 13

表 3-9 参数说明 14

表 3-10 返回值说明 14

表 3-11 参数说明 14

表 3-12 返回值说明 14

表 3-13 参数说明 15

表 3-14 返回值说明 15

表 3-15 参数说明 15

表 3-16 返回值说明 16

表 3-17 参数说明 16

表 3-18 返回值说明.....	16
表 3-19 参数说明	17
表 3-20 ril_operator_info 结构体说明.....	17
表 3-21 返回值说明.....	17
表 3-22 参数说明	17
表 3-23 返回值说明.....	18
表 3-24 参数说明	18
表 3-25 返回值说明.....	18
表 3-26 参数说明	19
表 3-27 返回值说明.....	19
表 3-28 参数说明	19
表 3-29 返回值说明.....	20
表 3-30 参数说明	20
表 3-31 返回值说明.....	20
表 3-32 参数说明	21
表 3-33 返回值说明.....	21
表 3-34 参数说明	21
表 3-35 返回值说明.....	21
表 3-36 参数说明	22
表 3-37 返回值说明.....	22
表 3-38 参数说明	22
表 3-39 返回值说明.....	23
表 3-40 错误码说明.....	34
表 3-41 错误码说明.....	38

1 概述

1.1 VoLTE 架构说明

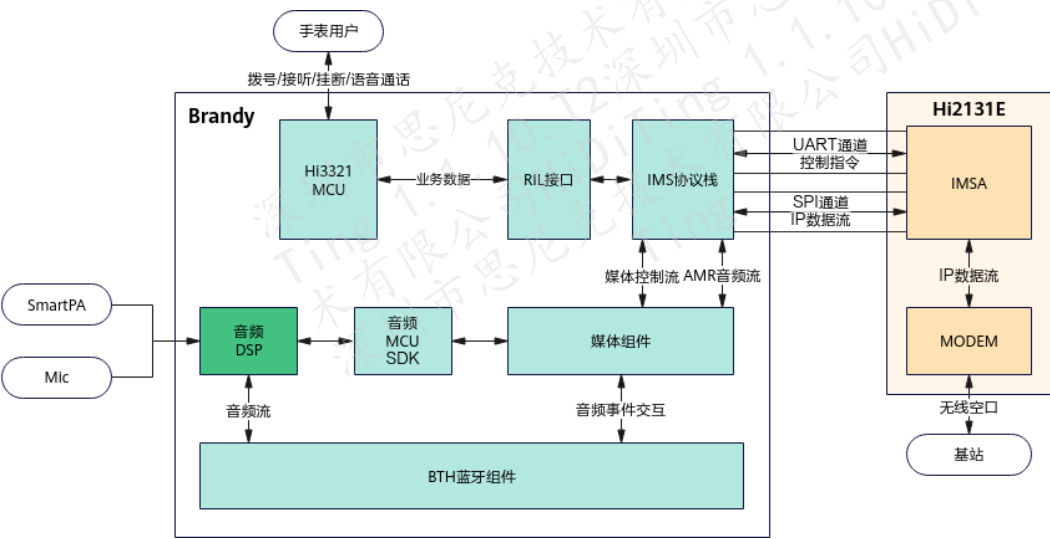
1.2 开发规范和约束

1.1 VoLTE 架构说明

VoLTE (Voice over LTE, 长期演进语音承载) 是 3GPP 标准定义的, 基于 IMS 网络的 LTE 语音解决方案。通过 IMS 网络, 移动运营商不仅可以无缝的继承传统的语音、短消息业务, 还可以将语音通话与丰富的增强功能相整合, 提供多样化的服务。本章节描述 VoLTE 场景下 Brandy 和 Hi2131E 芯片如何实现软件及硬件的二次开发, 以最终完成通话业务。

Hi2131E 作为支持 4G Cat1 的 Modem 蜂窝芯片, 提供 4G 入网数传能力。Brandy 作为穿戴芯片, 可提供语音编解码的处理能力。二者配合成为的 VoLTE 终端, 如图 1-1 所示。

图1-1 VoLTE 系统架构图



IMS 协议栈（VoLTE 基础功能的协议栈）主体代码位于 Brandy 芯片，Hi2131E 侧支持 4G 入网功能。和 Hi2131E 之间存在 UART 通路和 SPI 数据传输通路，UART 通路传输芯片间的控制指令，SPI 通路传输芯片间的数据信息。

表1-1 VoLTE 系统模块描述

组件名称	归属	说明
IMS 协议栈	Brandy	通过“libims.a”文件，集成编译在 Brandy SDK 版本中。
RIL 接口	Brandy	需要通过 Brandy 侧 RIL 头文件调用。
VoLTE 相关 AT 命令	Hi2131E	VoLTE 相关 AT 命令功能，需要通过接入 Hi2131E 侧的 UART 串口才可进行发送。

当前 VoLTE 终端提供的自能力如表 1-2 所示。

表1-2 IMS 子系统模块划分

VoLTE 功能分类	说明
基本音频呼叫	支持。
有卡紧急呼叫	支持。

VoLTE 功能分类	说明
呼叫方式	支持 Precondition 资源预留。
短信主被叫	支持。
语音编解码	仅支持 AMR-WB (mode-set=0-8) 编解码。
彩铃/漫游	支持 (IMS 网络需支持)。
SRVCC	不支持, 仅支持 4G 网络, 无法切换回落。
呼叫转移	不支持。
呼叫保持	不支持。
三方会议	不支持。

1.2 开发规范和约束

仅在 Brandy-diting 版本下支持和 Hi2131E 芯片的对接调试。

2 环境搭建

VoLTE 开发包含 Hi2131E 侧和 Brandy 侧两个部分，两个系统分别独立构建版本，通过 UART、SPI、GPIO 等连线建立通讯。

2.1 编译构建

2.2 DiTing 单板外接连线说明

2.3 VoLTE 拨号通话流程

2.1 编译构建

2.1.1 合入 VoLTE 功能修改

Brandy SDK 编译的编译宏配置及组件配置，可参考《HiDiTingV100 软件开发指南》文档“SDK 开发指导”章节。

- 与 VoLTE 功能相关的编译宏和组件配置，均已在 patch “build/config/target_config/brandy/config.py” 文件中开启，必须开启的宏如下：

```
'defines':  
[ 'FT_SINGLE_UART', 'SUPPORT_VOLTE', 'PRODUCT_EVB_VOLTE',  
  'CFG_FLASH_SIZE=16', 'SUPPORT_LWIP',  
  'SUPPORT_VOLTE_LOGFILE'
```

- 必须增加的组件名称如下：
'ram_component_set': ['volte_set'],
'ram_component': ['volte_at_service'],
- 音频相关 dsp_version 必须修改为如下内容：
'dsp_version': 'ultra'.

说明

Brandy 和 Hi2131E 对接场景下，本身支持的功能宏：单串口外设方案宏“FT_SINGLE_UART”，16M Flash 版本宏“CFG_FLASH_SIZE=16”。

2.1.2 Brandy 侧版本编译

Brandy 的编译命令，可参考《HiDiTingV100 软件开发指南》文档“编译构建环境”章节。

当前有 diting 专门的 target 版本支持 VoLTE 相关功能：

生成版本包名称：upload_app.fwpkg

生成目录：\tools\pkg\fwpkg\brandy

- pack_brandy_diting：

target 版本名称为 pack_brandy_diting (liteos + uikit + nandflash + mipi + psram + audio)。

编译命令：rm -rf output && python3 ./build.py pack_brandy_diting -nhso -release

说明

配置“-nhso -release”参数后，不会生成 database 打包文件，且不会生成“application.lst”反汇编文件，以减小 pkg 版本包生成时间。

2.1.3 Hi2131E 侧版本编译

当前版本中，Hi2131E 单板不提供二次开发界面和接口，功能接口和 AT 命令均已实现。软件版本已经烧录至单板中，直接使用即可。

2.2 DiTing 单板外接连线说明

现 DiTing 单板已将 Brandy 与 Hi2131E 芯片整合在同一单板上，以下对该单板外接串口小板以及连线进行说明。

表2-1 DiTing 单板外接连线说明

DiTing 单板模块	连接外设	说明
HSUART	串口小板	HSUART1、2、3 分别连接串口小板 TX、RX、GND。

DiTing 单板模块	连接外设	说明
		负责 Brandy 版本烧录。
UART2	串口小板	UART2 的 1、2、3 分别连接串口小板 TX、RX、GND。 负责 AT 口打印。
USB	Micro USB 线	负责 Hi2131E 的烧录以及 AT 日志。
USB-C	USB TYPE C	电源线。

2.3 VoLTE 拨号通话流程

Hi2131E 为实现正常的入网功能，需要一张可通过核心网鉴权的 USIM 卡。

2.3.1 通过 AT 命令拨打电话

AT 命令模块在 Hi2131E 侧，通过在 Hi2131E 侧调用相关 AT 命令，可以实现 VoLTE 通话。

主叫：

步骤 1 开机。

AT+CFUN=1

预期返回：OK

步骤 2 发起注册。

AT+CGATT=1

预期返回：OK

IMSSRVSTATUS: 0, 2 (请参见“3.2.9 IMS 服务状态更新指示：
[+IMSSRVSTATUS](#)”章节。)

步骤 3 拨打电话。

ATDxxx;

预期返回：OK

+ORIG: 1,0 (请参见“3.2.11 发起语音呼叫指示: +ORIG”章节。)

+CCALLSTATE: 1,0,1 (请参见“3.2.15 呼叫状态上报: +CALLSTATE”章节。)

+CCALLSTATE: 1,1,1 (收到网侧 100trying)

+CONF: 1 (请参见“3.2.12 UE 收到网络指示: +CONF”章节。)

+CHANNELINFO: 0,1 (请参见“3.2.16 主动上报语音带宽信息: +CHANNELINFO”章节。)

说明

无带内音信息, 需本地回铃音上报。此时需要自己制作铃声。

+CCALLSTATE: 1,2,1 (收到 180ringing。)

IRING (请参见“3.2.10 来电指示: IRING”章节。)

+CCALLSTATE: 1,3,1 (收到 invite 200OK, 呼叫已经接通。)

+CONN: 1,0 (请参见“3.2.13 呼叫接通指示: +CONN”章节。)

步骤 4 挂断电话。

ATH

预期返回: OK

+CCALLSTATE: 1,4,1 (通话结束)

+CEND: 1,100,0 (请参见“3.2.14 通话结束指示: +CEND”章节。)

----结束

被叫:

步骤 1 开机。

AT+CFUN=1

预期返回: OK

步骤 2 发起注册。

AT+CGATT=1

预期返回: OK

IMSSRVSTATUS: 0, 2 (请参见“3.2.9 IMS 服务状态更新指示: +IMSSRVSTATUS”章节。)

步骤 3 接听。

手动接听：ATA (请参见“3.2.2 接听来电：A”章节。)

自动接听：ATS0=x (请参见 3.2.4 设置自动接听：S0”章节。)

预期返回：OK

说明

自动接听命令建议在注册成功后执行，也可以在开机后执行。

手动接听则是在收到有来电提示 (+CALLSTATE: 1,5,1) 后执行。

步骤 4 挂断电话。

ATH

预期返回：OK

+CCALLSTATE: 1,4,1 (通话结束)

+CEND: 1,100,0

----结束

2.3.2 通过 RIL 接口拨打电话

RIL 接口存在于 Brandy 侧，通过在调用相关 RIL 接口，可以实现 VoLTE 通话。

主叫：

步骤 1 开机。

调用 ril_at_cmd_sync_get_info 接口 (请参见“3.1.5.2 同步 AT 命令发送”章节)。

说明

AT 命令：AT+CFUN=1

预期返回结果：OK

步骤 2 注册入网。

调用 ril_at_cmd_sync_get_info 接口。

说明

AT 命令：AT+CGATT=1

预期返回结果：OK

IMSSRVSTATUS: 0, 2

步骤 3 拨打电话。

调用 ril_set_call_orig_req 接口 (请参见 “[3.1.4.2 拨打电话](#)” 章节)。

步骤 4 挂断电话。

调用 ril_set_call_rel_req 接口 (请参见 “[3.1.4.4 挂断电话](#)” 章节)。

----结束

被叫:**步骤 1 开机。**

调用 ril_at_cmd_sync_get_info 接口 (请参见 “[3.1.5.2 同步 AT 命令发送](#)” 章节)。

步骤 2 注册入网。

调用 ril_at_cmd_sync_get_info 接口。

步骤 3 手动接听。

调用 ril_set_call_mt_accept_req 接口 (请参见 “[3.1.4.3 手动接听电话](#)” 章节)。

步骤 4 自动接听。

调用 ril_set_auto_answer 接口 (请参见 “[3.1.4.1 设置自动接听](#)” 章节)。

步骤 5 挂断电话。

调用 ril_set_call_rel_req 接口 (请参见 “[3.1.4.4 挂断电话](#)” 章节)。

----结束

3 对外接口描述

表3-1 RIL 接口实现方式

一级菜单	二级菜单	描述	实现方式
设备信息	软件版本号	获取 Modem 的软件版本信息。	核内交互
	IMEI 设备识别码	获取 Modem 设备的识别码。	核内交互
(U)SIM 卡信息	是否在位, Pin-block, Puk-block	获取 Modem (U)SIM 卡是否在位的信息, Pin 码和 PUK 码是否锁定状态。	核内交互
	IMSI	获取 (U)SIM 卡的 IMSI 信息。	核内交互
	本机号码显示	获取保存在 (U)SIM 中的本机号码。	核内交互
驻网状态	是否驻网	获取终端是否已经驻网。	核内交互
	显示运营商名称	获取终端驻网的运营商信息。	核内交互
	是否 IMS (VoLTE) 驻网	获取终端是否 IMS 驻网信息。	核内交互
	是否 PDP 驻网, 显示当前 IP 地址	获取终端是否 PDP 驻网, 并显示 IP 地址。	核内交互
	手动搜网, 并显示	触发手动搜网, 并返回搜	核间通信

一级菜单	二级菜单	描述	实现方式
	可选网络	网结果。	
通话设置	通话自动接听开关	设置是否通话自动接听。	核间通信
	拨打电话，如“10086”	通知 Modem 进行拨打电话。	核间通信
短信设置	显示短信中心号码	获取短信中心号码。	核内交互

- 3.1 RIL 相关接口
- 3.2 VoLTE 相关 AT 命令
- 3.3 VoLTE 相关 NV

3.1 RIL 相关接口

RIL 相关接口存在于 Brandy SDK 版本中，头文件路径：
protocol\ims\include\ril_interface.h

3.1.1 设备信息接口

3.1.1.1 获取软件版本号

- 函数原型：int32_t ril_get_software_version(char* ver, size_t size)
- 函数功能：获取 Hi2131E 软件版本号，核内交互。

表3-2 参数说明

参数	参数描述类型	参数结构类型	说明
ver	出参	char*	软件版本号，字符串格式。
size	入参	size_t	软件版本号长度。

表3-3 返回值说明

返回值	参数结构类型	说明
-----	--------	----

返回值	参数结构类型	说明
0	int32_t	获取 Hi2131E 软件版本号接口处理成功。
>0	int32_t	获取 Hi2131E 软件版本号接口处理失败。

3.1.1.2 获取 IMEI 设备识别号

- 函数原型：int32_t ril_get_imei(char* imei, size_t len)
- 函数功能：获取 Hi2131E 的 IMEI 号，核内交互。

表3-4 参数说明

参数	参数描述类型	参数结构类型	说明
imei	出参	char*	IMEI 号，字符串格式。
len	入参	size_t	IMEI 号长度。

表3-5 返回值说明

返回值	参数结构类型	说明
0	int32_t	获取 Hi2131E IMEI 号函数接口处理成功。
>0	int32_t	获取 Hi2131E IMEI 号函数接口处理失败。

3.1.1.3 设定 VoLTE 维测日志级别

- 函数原型：int32_t ril_set_user_log_level(ril_user_log_level level)
- 函数功能：设定 VoLTE 维测功能的日志级别。

表3-6 参数说明

参数	参数描述类型	参数结构类型	说明
level	入参	ril_user_log_level	RIL_USER_LEVEL_NULL：关闭日志。 RIL_USER_LEVEL_INFO：打印

参数	参数描述类型	参数结构类型	说明
			INFO 级别及更高级别日志。 RIL_USER_LEVEL_WARN：打印 WARN 级别及更高级别日志。 RIL_USER_LEVEL_ERROR：打 印 ERROR 级别及更高级别日志。 RIL_USER_LEVEL_ALL：打印所 有日志。

 说明

当前版本日志级别默认为 RIL_USER_LEVEL_ERROR 级别。

3.1.2 SIM 卡信息接口

3.1.2.1 判断 Hi2131E 的 SIM 卡是否在位

- 函数原型：int32_t ril_get_is_usim_in_position(bool *status);
- 函数功能：获取 Hi2131E 的(U)SIM 是否在位，核内交互。

表3-7 参数说明

参数	参数描述类型	参数结构类型	说明
status	出参	bool*	是否在位。

表3-8 返回值说明

返回值	参数结构类型	说明
0	int32_t	获取 Hi2131E 的 SIM 卡是否在位接 口处理成功。
>0	int32_t	获取 Hi2131E 的 SIM 卡是否在位接 口处理失败。

3.1.2.2 判断 Hi2131E 的 SIM 卡 PIN 码是否被锁

- 函数原型：int32_t ril_get_is_usim_pin_block(bool *status);
- 函数功能：获取 Hi2131E 的(U)SIM 卡 PIN 码是否被锁，核内交互。

表3-9 参数说明

参数	参数描述类型	参数结构类型	说明
status	出参	bool*	true：PIN 码被锁。 false：PIN 码没有被锁。

表3-10 返回值说明

返回值	参数结构类型	说明
0	int32_t	获取 Hi2131E 的(U)SIM 卡 PIN 码是否被锁接口处理成功。
>0	int32_t	获取 Hi2131E 的(U)SIM 卡 PIN 码是否被锁接口处理失败。

3.1.2.3 判断 Hi2131E 的 SIM 卡 PUK 码是否被锁

- 函数原型：int32_t ril_get_is_usim_puk_block(bool *status);
- 函数功能：获取 Hi2131E 的(U)SIM 卡 PUK 码是否被锁，核内交互。

表3-11 参数说明

参数	参数描述类型	参数结构类型	说明
status	出参	bool*	true：PUK 码被锁。 false：PUK 码没有被锁。

表3-12 返回值说明

返回值	参数结构类型	说明
-----	--------	----

返回值	参数结构类型	说明
0	int32_t	获取 Hi2131E 的(U)SIM 卡 PUK 码是否被锁接口处理成功。
>0	int32_t	获取 Hi2131E 的(U)SIM 卡 PUK 码是否被锁接口处理失败。

3.1.2.4 获取 Hi2131E 的 SIM 卡 IMSI 号

- 函数原型: int32_t ril_get_imsi(char* imsi, size_t len);
- 函数功能: 获取 Hi2131E 的(U)SIM 卡 IMSI 号, 核内交互。

表3-13 参数说明

参数	参数描述类型	参数结构类型	说明
imsi	出参	char*	IMSI 号, 字符串格式。
len	入参	size_t	IMSI 号长度。

表3-14 返回值说明

返回值	参数结构类型	说明
0	int32_t	获取 Hi2131E 的(U)SIM 卡 IMSI 号接口处理成功。
>0	int32_t	获取 Hi2131E 的(U)SIM 卡 IMSI 号接口处理失败。

3.1.2.5 获取 Hi2131E 的 SIM 卡 msisdn

- 函数原型: int32_t ril_get_usim_msisdn(char* msisdn, size_t len);
- 函数功能: 获取 Hi2131E 的(U)SIM 卡 msisdn 信息, 核内交互。

表3-15 参数说明

参数	参数描述类型	参数结构类型	说明
imsi	出参	char*	msisdn, 字符串格式。

参数	参数描述类型	参数结构类型	说明
len	入参	size_t	msisdn 长度。

表3-16 返回值说明

返回值	参数结构类型	说明
0	int32_t	获取 Hi2131E 的(U)SIM 卡 msisdn 信息接口处理成功。
>0	int32_t	获取 Hi2131E 的(U)SIM 卡 msisdn 信息接口处理失败。

3.1.3 驻网状态接口

3.1.3.1 获取驻网状态

- 函数原型：int32_t ril_get_radio_reg_status(uint8_t *status);
- 函数功能：获取 Hi2131E 的驻网状态，核内交互。

表3-17 参数说明

参数	参数描述类型	参数结构类型	说明
status	出参	uint8_t*	0： EMM detached。 1： EMM attached。

表3-18 返回值说明

返回值	参数结构类型	说明
0	uint8_t	获取 Hi2131E 的驻网状态接口处理成功。
>0	uint8_t	获取 Hi2131E 的驻网状态接口处理失败。

3.1.3.2 显示运营商信息

- 函数原型：int32_t ril_get_radio_operator_info(ril_operator_info *operator_info);
- 函数功能：显示 Hi2131E 的运营商信息，核内交互。

表3-19 参数说明

参数	参数描述类型	参数结构类型	说明
operator_info	出参	ril_operator_info*	运营商信息。

表3-20 ril_operator_info 结构体说明

字段	参数结构类型	说明
numeric	char[20]	-
name_short	char[20]	-
name_long	char[20]	-

表3-21 返回值说明

返回值	参数结构类型	说明
0	int32_t	显示运营商信息接口处理成功。
>0	int32_t	显示运营商信息接口处理失败。

3.1.3.3 获取 IMS 驻网状态

- 函数原型：int32_t ril_get_radio_ims_reg(bool *status);
- 函数功能：获取 IMS 的驻网状态，核内交互。

表3-22 参数说明

参数	参数描述类型	参数结构类型	说明
status	出参	bool*	IMS 注册状态。 true：已注册；

参数	参数描述类型	参数结构类型	说明
			false：未注册。

表3-23 返回值说明

返回值	参数结构类型	说明
0	int32_t	获取 IMS 的驻网状态接口处理成功。
>0	int32_t	获取 IMS 的驻网状态接口处理失败。

3.1.3.4 是否 PDP 驻网，显示 IP 地址

- 函数原型：int32_t ril_get_radio_pdp_info(respond_callback_t rsp_callback);
- 函数功能：获取 PDP 地址，核内交互（相当于执行：AT+CGPADDR）。

表3-24 参数说明

参数	参数描述类型	参数结构类型	说明
rsp_callback	入参	respond_callback_t	回调函数： typedef int32_t (*respond_callback_t)(uint8_t* data, size_t len);

表3-25 返回值说明

返回值	参数结构类型	说明
0	int32_t	获取 PDP 接口处理成功。
>0	int32_t	获取 PDP 接口处理失败。

3.1.3.5 手动搜网，并显示可选网络

- 函数原型：int32_t ril_get_radio_search_start(respond_callback_t rsp_callback);

- 函数功能：获取在网络中搜索到的 PLMN 清单，核内交互（相当于执行：AT+COPS=?）。

表3-26 参数说明

参数	参数描述类型	参数结构类型	说明
rsp_callback	入参	respond_callback_t	回调函数： typedef int32_t (*respond_callback_t) (uint8_t* data, size_t len);

表3-27 返回值说明

返回值	参数结构类型	说明
0	int32_t	获取 PLMN 清单接口处理成功。
>0	int32_t	获取 PLMN 清单接口处理失败。

3.1.4 通话接口

3.1.4.1 设置自动接听

- 函数原型：int32_t ril_set_auto_answer(uint8_t time_s);
- 函数功能：设置自动应答，核内交互（相当于执行：ATS0=）。

表3-28 参数说明

参数	参数描述类型	参数结构类型	说明
time_s	入参	uint8_t	S0 定时器时长。 0：关闭自动接听； 1~255：启动自动应答 的时间，单位：s。

表3-29 返回值说明

返回值	参数结构类型	说明
0	int32_t	设置自动接听接口处理成功。
>0	int32_t	设置自动接听接口处理失败。

3.1.4.2 拨打电话

- 函数原型: int32_t ril_set_call_orig_req(const char* call_str, respond_callback_t rsp_callback);
- 函数功能: 拨打电话, 核内交互 (相当于执行: ATDxxx;)。

表3-30 参数说明

参数	参数描述类型	参数结构类型	说明
call_str	入参	char*	拨打的电话号码。
rsp_callback	入参	respond_callback_t	回调函数: typedef int32_t (*respond_callback_t)(uint8_t* data, size_t len);

表3-31 返回值说明

返回值	参数结构类型	说明
0	int32_t	拨打电话接口处理成功。
>0	int32_t	拨打电话接口处理失败。

3.1.4.3 手动接听电话

- 函数原型: int32_t ril_set_call_mt_accept_req(respond_callback_t rsp_callback);
- 函数功能: 手动接听电话, 核内交互 (相当于执行: ATA)。

表3-32 参数说明

参数	参数描述类型	参数结构类型	说明
rsp_callback	入参	respond_callback_t	回调函数： typedef int32_t (*respond_callback_t)(uint8_t* data, size_t len);

表3-33 返回值说明

返回值	参数结构类型	说明
0	int32_t	接听电话接口处理成功。
>0	int32_t	接听电话接口处理失败。

3.1.4.4 挂断电话

- 函数原型：int32_t ril_set_call_rel_req(respond_callback_t rsp_callback);
- 函数功能：挂断电话，核内交互（相当于执行：ATH）。

表3-34 参数说明

参数	参数描述类型	参数结构类型	说明
rsp_callback	入参	respond_callback_t	回调函数： typedef int32_t (*respond_callback_t)(uint8_t* data, size_t len);

表3-35 返回值说明

返回值	参数结构类型	说明
0	int32_t	挂断电话接口处理成功。
>0	int32_t	挂断电话接口处理失败。

3.1.5 AT 指令通道

Hi2131E 侧 AT 命令遵循 3GPP TS 27.007 和 27.005 规范。

3.1.5.1 AT 指令发送

- 函数原型：int32_t ril_at_cmd_exc(const char *str, respond_callback_t rsp_callback);
- 函数功能：执行 AT 命令，核内交互。

表3-36 参数说明

参数	参数描述类型	参数结构类型	说明
str	入参	char*	AT 命令字符串。
rsp_callback	入参	respond_callback_t	回调函数： typedef int32_t (*respond_callback_t)(uint8_t* data, size_t len);

表3-37 返回值说明

返回值	参数结构类型	说明
0	int32_t	执行 AT 命令接口处理成功。
>0	int32_t	执行 AT 命令接口处理失败。

3.1.5.2 同步 AT 命令发送

- 函数原型：int32_t ril_at_cmd_sync_get_info(const char *at_str, uint8_t *data, uint16_t size);
- 函数功能：同步执行 AT 命令，核内交互。

表3-38 参数说明

参数	参数描述类型	参数结构类型	说明
at_str	入参	char*	AT 命令字符串。

参数	参数描述类型	参数结构类型	说明
data	出参	uint8_t*	AT 命令返回的结果字符串。
size	入参	uint16_t	AT 命令结果字符串的长度。

表3-39 返回值说明

返回值	参数结构类型	说明
0	int32_t	执行同步 AT 命令接口处理成功。
>0	int32_t	执行同步 AT 命令接口处理失败。

3.2 VoLTE 相关 AT 命令

3.2.1 发起呼叫请求：D

命令描述

发起一个呼叫。

命令语法

命令	返回结果	示例
D[<dial_string>][I][G];	OK 错误情况： +CME ERROR: <err>	atd13900000000; OK atd13900000000i; OK atd13900000000l; OK

参数描述

参数	描述
<dial_string>	字符串类型。呼叫号码。
[I]	如果命令中带有 I，表示在这次呼叫中禁止显示主叫号码； 如果命令中带有 i，表示在这次呼叫中允许显示主叫号码。
[G]	暂不支持。一般表示在这次呼叫中使能 CUG 补充业务。

3.2.2 接听来电：A

命令描述

接听当前的来电。

命令语法

命令	返回结果	示例
A	OK 错误情况： +CME ERROR: <err>	ATA OK

3.2.3 挂断电话：H

命令描述

挂断电话。

命令语法

命令	返回结果	示例
H[<value>]	OK 错误情况： +CME ERROR: <err>	ATH OK

参数描述

参数	描述
<value>	整型值，仅支持取值为 0。 取其他值时，响应结果均为 ERROR。

3.2.4 设置自动接听：S0

命令描述

设置自动应答。

命令语法

命令	返回结果	示例
S0=<n>	OK 错误情况： +CME ERROR: <err>	ATS0=5 OK
S0?	<CR><LF><n><CR><LF> <CR><LF>OK<CR><LF>	ATS0? 5 OK
S0=?	<CR><LF>S0: (list of supported <n>s) <CR><LF> <CR><LF>OK<CR><LF>	ATS0=? S0: (0-255) OK

参数描述

参数	描述
<n>	整型值，默认值为 0。 0：关闭自动应答； 1 ~ 255：启动自动应答的时间，单位：s。 注：不同的核心网规定的振铃超时时间不同，设置的自动应答时间

参数	描述
	过长可能会导致核心网“Altering timeout”异常，从而拒绝呼叫。 IMS 核心网网元通常配置被叫侧呼叫振铃超时定时器时长为 60s，建议此命令配置时长不要超过该值。

3.2.5 查询当前呼叫信息：+CLCC

命令描述

列出当前所有的呼叫。

命令语法

命令	返回结果	示例
+CLCC	+CLCC: <id1>,<dir>,<stat>,<mode>,<mpty>[, <number>,<type>[,<alpha>]] <id2>,<dir>,<stat>,<mode>,<mpty>[, <number>,<type>	AT+CLCC +CLCC: 1,0,0,0,0,"888",129 OK
+CLCC=?	<CR><LF><n><CR><LF> <CR><LF>OK<CR><LF>	AT+CLCC=? OK

参数描述

参数	描述
<idx>	整型值，呼叫标识。
<dir>	整型值，呼叫方向。 0：主叫； 1：被叫。
<state>	整型值，呼叫状态。 0：激活状态；

参数	描述
	1: 呼叫保持状态 2: 主叫, 拨号状态; 3: 主叫, 回铃音状态; 4: 被叫, 来电状态;
<mode>	整型值, 呼叫模式。(当前仅支持语音呼叫模式。) 0: 语音呼叫。
<mpty>	整型值, 多方通话状态。 0: 不在多方通话中。
<number>	字符串类型, 电话号码。 主叫侧显示被叫号码, 被叫侧显示主叫号码。
<type>	号码地址类型, 整型值。 129: 普通号码; 145: 国际号码 (以 “+” 开头) 。

3.2.6 查询 IMS 域注册状态: +CIREG

命令描述

查询当前 IMS 域网络注册状态以及设置状态上报方式。

命令语法

命令	返回结果	示例
+CIREG=[<n>]	OK 错误情况: +CME ERROR: <err>	AT+CIREG=1 OK
+CIREG?	+CIREG: <n>,<reg_info>[,<ext_info>] 错误情况: +CME ERROR: <err>	AT+CIREG? +CIREG:0,1,5 (IMS 已注册且支持语音和短信) OK

命令	返回结果	示例
+CIREG=?	+CIREG: (list of supported <n>s) OK	AT+CIREG=? +CIREG: (0-2) OK

参数描述

参数	描述
<n>	整型值，主动上报方式，默认值为 0。 0：禁止+CIREGU 的主动上报； 1：使能+CIREGU: <reg_info>的主动上报； 2：使能+CIREGU: <reg_info>[,<ext_info>]的主动上报。
<reg_info>	整型值，注册状态。 0：未注册 IMS 网络； 1：已注册 IMS 网络。
<ext_info>	整型值，IMS 域能力值，取值范围[1,0xFFFFFFFF]，每个 bit 位表示不同的能力。 1：RTP-based transfer of voice； 2：RTP-based transfer of text； 4：SMS using IMS functionality； 8：RTP-based transfer of video。 例如，等于 5 的时候，表示支持 RTP-based transfer of voice 和 SMS using IMS functionality。 注：IMS 已注册才显示该值

3.2.7 主动上报 IMS 域注册状态：+CIREGU

命令描述

主动上报当前 IMS 域网络注册状态，能否主动上报受+CIREG 命令控制。

命令语法

命令	返回结果	示例
-	+CIREGU: <reg_info>[,<ext_info>]	+CIREGU: 1,5 OK

参数描述

参数	描述
<reg_info>	整型值，注册状态。 0：未注册 IMS 网络； 1：注册了 IMS 网络。
<ext_info>	整型值，IMS 域能力值，取值范围[1,0xFFFFFFFF]，每个 bit 位表示不同的能力。 1: RTP-based transfer of voice; 2: RTP-based transfer of text; 4: SMS using IMS functionality; 8: RTP-based transfer of video。 例如，等于 5 时，表示支持 RTP-based transfer of voice 和 SMS using IMS functionality。

3.2.8 查询 IMS 域的虚拟号码：+VOLTEIMPU

命令描述

该命令用于查询 IMS 域的 IMPU 信息以及虚拟号码。

命令语法

命令	返回结果	示例
+VOLTEIMPU	<CR><LF>OK<CR><LF> <CR><LF>+VOLTEIMPU: <IMPU>[,<Virtual_Number >]<C	AT+VOLTEIMPU +VOLTEIMPU: abcd,123456 OK

命令	返回结果	示例
	R><LF> 错误情况: <CR><LF>+CME ERROR: <err><CR><LF>	
+VOLTEIMPU?	OK	AT+VOLTEIMPU? OK
+VOLTEIMPU= ?	OK	AT+VOLTEIMPU=? OK

参数描述

参数	说明
<IMPU>	字符串类型，IMPU 信息，取值范围 0 ~ 128Byte。
<Virtual_Number>	字符串类型，网络分配的虚拟电话号码，最大长度为 127Byte。

3.2.9 IMS 服务状态更新指示：+IMSSRVSTATUS

命令描述

IMS 服务状态或者注册状态发生变化的主动上报。

命令语法

命令	示例
+IMSSRVSTATUS: <sms_srv_status>,<voip_srv_status>	+IMSSRVSTATUS: 2,0

参数描述

参数	描述
<sms_srv_status>	整型值，IMS 短信的服务状态：

参数	描述
	0: IMS SMS service is not available; 1: IMS SMS is in limited service; 2: IMS SMS is in full service.
<voip_srv_status>	整型值, IMS VoIP 的服务状态: 0: IMS VoIP service is not available; 1: IMS VoIP is in limited service; 2: IMS VoIP is in full service.

3.2.10 来电指示: IRING

命令描述

IMS 域来电指示。

命令语法

命令	示例
IRING	有 IMS 域来电 IRING。

3.2.11 发起语音呼叫指示: +ORIG

命令描述

主动上报命令, 在用户发起电路域呼叫时, 通知用户 UE 正在向网络发起呼叫。

命令语法

命令	示例
+ORIG: <call_id>,<call_type>	UE 向网络发起一个语音呼叫 +ORIG: 1,0。

参数描述

参数	说明
call_id	整型值，呼叫 ID。
call_type	整型值，呼叫类型。 0：语音呼叫； 9：紧急呼叫。

3.2.12 UE 收到网络指示：+CONF

命令描述

主动上报命令，通知用户发起呼叫后，UE 收到了网络接通指示。

命令语法

命令	示例
+CONF: <call_id>	UE 收到网络接通指示 +CONF: 1。

参数描述

参数	说明
call_id	整型值，呼叫 ID。

3.2.13 呼叫接通指示：+CONN

命令描述

主动上报命令，当呼叫接通后，UE 通知用户呼叫已经接通。

命令语法

命令	示例
+CONN:<call_id>, <call_type>	语音呼叫已经接通 +CONN:1,0。

参数描述

参数	说明
call_id	整型值，呼叫 ID。
call_type	整型值，呼叫类型。 0：语音呼叫； 9：紧急呼叫。

3.2.14 通话结束指示：+CEND

命令描述

主动上报命令，在通话结束后，UE 通知用户通话结束原因和通话时长。

命令语法

命令	示例
+CEND: <call_id>, <duration>, <cause>	通话结束 +CEND: 1,10,0

参数描述

参数	说明
call_id	整型值，呼叫 ID。
duration	整型值，通话时长，单位：s。

参数	说明
cause	整型值，呼叫错误原因值；如表 3-40 所示。

表3-40 错误码说明

错误码	数值	说明
IMSA_CALL_CAUSE_SUCESS	0	正常挂断。
IMSA_CALL_CAUSE_IMSA_ERROR	1	IMSA 特殊无服务异常。 IMSA_INTRA_CALL_SRV_STATUS_IND 消息中 no_srv_cause 不是 IMSA_CALL_NO_SRV_CAUSE_MEDIA_PDP_RELEASE、 IMSA_CALL_NO_SRV_CAUSE_SIP_PDP_ERR、 IMSA_CALL_NO_SRV_CAUSE_REG_ERR、 IMSA_CALL_NO_SRV_CAUSE EMC_SIP_PDP_NO_PCSCF。
IMSA_CALL_CAUSE_IMSA_TCALL_TIMEOUT	2	IMSA Tcall 超时。
IMSA_CALL_CAUSE_IMSA_WAIT_RES_TIMEOUT	3	等待资源预留成功定时器超时。
IMSA_CALL_CAUSE_IMS_EMCMC_REG_FAIL	4	紧急注册失败引起紧急呼叫失败。
IMSA_CALL_CAUSE_IMS_INNER_ERROR	5	IMS 协议栈内部异常，IMS 侧 sip_status_code<100。
IMSA_CALL_CAUSE_IMSA_STRM_RTP_BREAK	6	IMS 侧 sip_status_code=100。
IMSA_CALL_CAUSE_IMSA_IMS_ERROR	7	IMS 侧 100<sip_status_code<300 或者 sip_status_code≥700。
IMSA_CALL_CAUSE_NORMAL_DIAL_ERROR	8	IMSA 普通呼叫 dial 请求创建失败。
IMSA_CALL_CAUSE_EMCMC_DIAL_ERROR	9	IMSA 紧急呼叫 dial 请求创建失败。

错误码	数值	说明
IMSA_CALL_CAUSE_SIGNAL_PDP_ERR	10	默认承载被释放。
IMSA_CALL_CAUSE_MEDIA_PDP_ERR	11	专有承载被释放。
IMSA_CALL_CAUSE_EMM_EST_ERR	12	EMM 建链失败引起呼叫失败。
300 ~ 700	300 ~ 700	IMS 侧 $300 \leq \text{sip_status_code} < 700$ 。

3.2.15 呼叫状态上报：+CALLSTATE

命令描述

主动上报命令，呼叫过程中，主动上报呼叫状态的变化。

命令语法

命令	示例
+CALLSTATE: <call_id>, <state>, <domain>	有 IMS 域来电 +CALLSTATE: 1,5,1

参数描述

参数	说明
call_id	整型值，呼叫 ID。
state	整数型，指示对应 call_id 的呼叫状态。 0: Call originate (MO)，向网络发起呼叫； 1: Call is Proceeding，呼叫发起，网络已经开始处理； 2: Alerting，振铃； 3: Connected，呼叫已经接通； 4: Released，呼叫被释放；

参数	说明
	5: InComing, 有来电; 6: Waiting, 有一个等待的来电; 7: Hold, 呼叫保持; 8: Retrieve, 呼叫恢复。
domain	整型值, 语音电话的呼叫域。 0: CS 域语音电话; 1: IMS 域语音电话。

3.2.16 主动上报语音带宽信息: +CHANNELINFO

命令描述

主动上报命令, 呼叫过程中, 主动上报语音带宽的信息。

命令语法

命令	示例
+CHANNELINFO: <status>,<domain>	IMS 域语音电话, 当前语音处于宽带语音时, 主动上报 +CSCHANNELINFO: 2,1

参数描述

参数	说明
status	整型值, 语音信道的状态。 0: 振铃, 无带内音信息 (本地回铃音上报); 1: 带内音可用, 窄带语音 (网侧携带铃音); 2: 带内音可用, 宽带语音 (网侧携带铃音)。
domain	整型值, 语音电话的呼叫域。 0: CS 域语音电话 (本产品无 CS);

参数	说明
	1: IMS 域语音电话。

3.2.17 上报 IMS 注册失败原因：+IMSREGERR

命令描述

该命令用于上报 IMS 注册失败原因。

命令语法

命令	示例
+IMSREGERR: <err_type>, <err_code>,<sip_reason>	通话结束 +IMSREGERR: 1,3,

参数描述

参数	说明
err_type	整型值，错误类型。 0: PDN 建立失败; 1: IMS 注册失败。
err_code	整数型，失败原因值。 当 err_type 为 0 时: PDN 建立失败原因。 当 err_type 为 1 时: 0: 注册模块命令执行成功（不会出现）; 1: 注册模块命令执行失败; 2: 网侧 SIP 拒绝，具体原因见 sip_reason; 3: 发送或接收 SIP 报文超时; 4: 发送或接收 SIP 报文时，底层传输发生错误; 5: 注册成功后，网侧发起的去注册; 6: 没有可以再使用的注册地址对;

参数	说明
	7：注册被禁止（暂时仅作预留，不会出现）； 8：周期注册，被网侧拒绝； 300～700：IMS 失败原因值。
sip_reason	字符串值类型，网侧 SIP 消息拒绝携带的原因值。

3.2.18 IMS 域被叫处理状态信息主动上报：+IMSMTRPT

命令描述

该命令用于上报 IMSA 开始处理被叫。

命令语法

命令	示例
+IMSMTRPT: <number>,<status>,<cause>	IMSA 开始处理被叫主动上报： +IMSMTRPT: +8618000009193,0,0

参数描述

参数	说明
number	字符串，被叫来电号码。
status	整型值，状态。 0：收到被叫，开始处理被叫过程； 1：被叫异常，未收到 RING 就结束。
cause	整型值，呼叫错误原因值；如表 3-41 所示。

表3-41 错误码说明

错误码	数值	说明
-----	----	----

错误码	数值	说明
IMSA_CALL_CAUSE_SUCCESS	0	正常挂断。
IMSA_CALL_CAUSE_IMSA_ERROR	1	IMSA 特殊无服务异常。 IMSA_INTRA_CALL_SRV_STATUS_IND 消息中 no_srv_cause 不是 IMSA_CALL_NO_SRV_CAUSE_MEDIA_PDP_RELEASE, IMSA_CALL_NO_SRV_CAUSE_SIP_PDP_ERR, IMSA_CALL_NO_SRV_CAUSE_REG_ERR , IMSA_CALL_NO_SRV_CAUSE_EMCC_SIP_PDP_NO_PCSCF。
IMSA_CALL_CAUSE_IMSA_TCALL_TIMEOUT	2	IMSA Tcall 超时。
IMSA_CALL_CAUSE_IMSA_WAIT_RES_TIMEOUT	3	等待资源预留成功定时器超时。
IMSA_CALL_CAUSE_IMS_EMCC_REG_FAIL	4	紧急注册失败引起紧急呼叫失败。
IMSA_CALL_CAUSE_IMS_INNER_ERROR	5	IMS 协议栈内部异常, IMS 侧 sip_status_code<100。
IMSA_CALL_CAUSE_IMSA_STRM_RTP_BREAK	6	IMS 侧 sip_status_code=100。
IMSA_CALL_CAUSE_IMSA_IMS_ERROR	7	IMS 侧 100<sip_status_code<300 或者 sip_status_code≥700。
IMSA_CALL_CAUSE_NORMAL_DIAL_ERROR	8	IMSA 普通呼叫 dial 请求创建失败。
IMSA_CALL_CAUSE_EMCC_DIAL_ERROR	9	IMSA 紧急呼叫 dial 请求创建失败。
IMSA_CALL_CAUSE_SIGNAL_PDP_ERR	10	默认承载被释放。
IMSA_CALL_CAUSE_MEDIA_PDP_ERR	11	专有承载被释放。
IMSA_CALL_CAUSE_EMM_EST_ERR	12	EMM 建链失败引起呼叫失败。
300 ~ 700	300 ~	IMS 侧 300≤sip_status_code<700。

错误码	数值	说明
	700	

3.2.19 关闭 IMS 功能开关：+IMSSWITCH

命令描述

配置开启/关闭 IMS 协议栈。

命令语法

命令	返回结果	示例
+IMSSWITCH=[<n>]	OK 错误情况： +CME ERROR: <err>	AT+IMSSWITCH=1 OK
+IMSSWITCH=?	+IMSSWITCH: (list of supported <n>s) OK	AT+IMSSWITCH=? +IMSSWITCH: (0-1) OK

参数描述

参数	描述
<n>	整型值，默认值为 1。 0：关闭 IMS 协议栈； 1：打开 IMS 协议栈。

3.3 VoLTE 相关 NV

VoLTE 支持的 NV 配置，均存在 Hi2131E 上，可以通过 Hi2131E 的 DebugKits 工具对这些 NV 进行配置修改，建议 VoLTE 相关的 NV 全部保持默认值配置。

定义路径

- 公共配置文件：
middleware/chips/cat1/nv/nv_config/acore/cfg/acore/common.json
- Hi2131E 独有配置文件：
middleware/chips/cat1/nv/nv_config/acore/cfg/acore/acore_ims.json
- 客户自定义配置文件：
middleware/chips/cat1/nv/nv_config/acore/cfg/acore/customer.json

定义约束

- 用户新定义的 module_id 应位于 1 ~ 255 之间。
- 用户新定义的 module_id 应按顺序编号，避免重复。

模块说明

参数	说明
模块 ID	31
模块名	IMSA
模块描述	IMS 适配层
模块类型	Custom

模块 Key

IMS_SIP_CONFIG	
Key ID	0x5001
Key 状态	alive
Key 描述	IMS NV 配置。
结构类型	<pre>#define IMS_PARM_STRING_SIZE 129 #define NUM_TWO_SIZE 2 typedef enum { IMS_PARM_URI_FORMAT_TYPE_DEFAULT = 0, /* default */ IMS_PARM_URI_FORMAT_TYPE_SIP, /* sip */ IMS_PARM_URI_FORMAT_TYPE_TEL, /* tel */ IMS_PARM_URI_BUTT = 0xFF, } ims_parm_uri_format_type; typedef enum { IMS_PARM_TRANSPORT_TYPE_UDP = 0,</pre>

IMS_SIP_CONFIG	
	<pre> IMS_PARM_TRANSPORT_TYPE_TCP, IMS_PARM_TRANSPORT_TYPE_BUTT = 0xFF, } ims_parm_transport_type; typedef struct { ims_parm_transport_type transport_type; ims_parm_uri_format_type uri_format; uint8_t header_compact_enabled; uint8_t forking_flag; uint32_t server_port; uint32_t timer_sip_reg_value; uint32_t subs_reg_expires; uint32_t rsv1; uint32_t rsv2; uint32_t rsv3; uint32_t rsv4; uint32_t sip_socket_keep_alive_value; int8_t user_agent[IMS_PARM_STRING_SIZE]; int8_t timer_reg; int8_t sms_format_timer_dereg[NUM_TWO_SIZE]; uint32_t tcp_threshold; } ims_parm_sip; </pre>
默认值	<pre> transport_type =IMS_PARM_TRANSPORT_TYPE_UDP uri_format =IMS_PARM_URI_FORMAT_TYPE_DEFAULT header_compact_enabled =0 forking_flag =0(不支持修改) server_port =5060 timer_sip_reg_value =600000 subs_reg_expires =2000 rsv1 =16000 (不支持修改) rsv2 =17000 (不支持修改) rsv3 =128000 (不支持修改) rsv4 =128000 (不支持修改) sip_socket_keep_alive_value =0 (不支持修改) user_agent[IMS_PARM_STRING_SIZE] ={0} (不支持修改) timer_reg =0 (不支持修改) sms_format_timer_dereg[NUM_TWO_SIZE] ={2,0} (不支持修改) tcp_threshold =16500 </pre>
备注	-

IMS_PARM_VOIP	
Key ID	0x5002
Key 状态	alive

IMS_PARM_VOIP	
Key 描述	IMS VoIP 配置。
结构类型	<pre>typedef enum { IMS_PARM_SESSION_REFRESH_TYPE_UAC = 0x00, /* Client Refresh */ IMS_PARM_SESSION_REFRESH_TYPE_UAS, /* Server Refresh */ IMS_PARM_SESSION_REFRESH_TYPE_BUTT = 0xFF, } ims_parm_session_refresh_type; typedef enum { IMS_PARM_SESSION_REFRESH_METHOD_UPDATE = 0x00, /* Update */ IMS_PARM_SESSION_REFRESH_METHOD_REINVITE, /* Reinvite */ IMS_PARM_SESSION_REFRESH_METHOD_BUTT = 0xFF, } ims_parm_session_refresh_method; typedef struct { uint8_t session_timer_enabled; uint8_t pre_condition_enable; ims_parm_session_refresh_type session_refresh_type; ims_parm_session_refresh_method session_refresh_method; uint16_t session_expires; uint8_t rtp_inactive_timer[2]; } ims_parm_voip;</pre>
默认值	<pre>session_timer_enabled =1 (不支持修改) pre_condition_enable =1 session_refresh_type =IMS_PARM_SESSION_REFRESH_TYPE_UAC (不支持修改) session_refresh_method =IMS_PARM_SESSION_REFRESH_METHOD_UPDATE (不支持修改) session_expires =1800 (不支持修改) rtp_inactive_timer[2] ={10,1} (不支持修改)</pre>
备注	-

IMS_PARM_VOIP_CODE	
Key ID	0x5003
Key 状态	alive
Key 描述	IMS VoIP 语音编码配置。
结构类型	<pre>typedef enum { IMS_PARM_AUDIO_CAP_AMR_NB_AND_WB = 0x00, /* AMR- NB and AMR-WB (AMR-NB preferred) */ IMS_PARM_AUDIO_CAP_AMR_NB_AND_WB, /* AMR-</pre>

IMS_PARM_VOIP_CODE	
	<pre> WB and AMR-NB (AMR-WB preferred) */ IMS_PARM_AUDIO_CAP_AMR_NB, /* AMR- NB */ IMS_PARM_AUDIO_CAP_AMR_WB, /* AMR- WB */ IMS_PARM_AUDIO_CAP_EVS_AND_AMR_NB_AND_WB, /* AMR-NB and AMR-WB and EVS (EVS preferred) */ IMS_PARM_AUDIO_CAP_EVS = 5, IMS_PARM_AUDIO_CAP_TYPE_BUTT } ims_parm_audio_cap; typedef enum { IMS_PARM_AUDIO_CODE_MODE_OCTECT_ALIGN = 0x00, /* Octet-aligned Mode */ IMS_PARM_AUDIO_CODE_MODE_BAND_EFFICIENT, /* Bandwidth-Efficient Mode */ IMS_PARM_AUDIO_CODE_MODE_EF_AND_OC, /* bandwidth-efficient and octet-aligned Mode */ IMS_PARM_AUDIO_CODE_MODE_OC_AND_EF, /* octet-aligned and bandwidth-efficient Mode */ IMS_PARM_AUDIO_CODE_TYPE_BUTT } ims_parm_audio_code_mode; typedef struct { ims_parm_audio_cap comm_audio_cap; uint8_t set_nb_amr_mode_flag; uint8_t set_wb_amr_mode_flag; ims_parm_audio_cap nr_audio_cap; uint32_t nb_amr_mode_set; uint32_t wb_amr_mode_set; ims_parm_audio_code_mode nb_amr_audio_code_mode; ims_parm_audio_code_mode wb_amr_audio_code_mode; uint16_t ptimer_value; uint8_t video_resolution; // 以下参数为 保持和 IMS 一致 uint8_t h264_level; uint8_t video_fps; uint8_t cvo_flag; uint32_t video_bitrate; } ims_parm_voip_code; </pre>
默认值	<pre> comm_audio_cap =IMS_PARM_AUDIO_CAP_AMR_WB_AND_NB set_nb_amr_mode_flag =0 set_wb_amr_mode_flag =0 nr_audio_cap =IMS_PARM_AUDIO_CAP_AMR_NB_AND_WB nb_amr_mode_set =0 wb_amr_mode_set =0 nb_amr_audio_code_mode =IMS_PARM_AUDIO_CODE_MODE_EF_AND_OC wb_amr_audio_code_mode =IMS_PARM_AUDIO_CODE_MODE_EF_AND_OC ptimer_value =20 video_resolution =0 h264_level =1 </pre>

IMS_PARM_VOIP_CODE	
	video_fps =30
	cvo_flag =0
	video_bitrate =640000
备注	不支持修改。

IMS_PARM_SS_CONF	
Key ID	0x5004
Key 状态	alive
Key 描述	IMS VoIP 语音编码配置。
结构类型	<pre>#define IMS_PARM_STRING_SIZE 129 typedef struct { uint8_t conf_factory_uri[IMS_PARM_STRING_SIZE]; uint8_t reject_with_486[3]; } ims_parm_ss_conf;</pre>
默认值	<pre>conf_factory_uri = {0} reject_with_486 = {0}</pre>
备注	不支持修改。

IMS_PARM_SS_SECURITY	
Key ID	0x5005
Key 状态	alive
Key 描述	IMS IPSEC 配置。
结构类型	<pre>typedef struct { uint8_t null; uint8_t aes; uint8_t des; (不支持修改) uint8_t rsv[1]; } ealgo_capability; typedef struct { uint8_t md5; (不支持修改) uint8_t sha; uint8_t rsv[2]; } algo_capability; typedef struct { uint8_t srtp_flag; (不支持修改) uint8_t sock_support_linger[3]; (不支持修改) ealgo_capability ealgo;</pre>

IMS_PARM_SS_SECURITY	
	<pre> algo_capability algo; } ims_parm_security; </pre>
默认值	<pre> srtp_flag =0 sock_support_linger[3] ={2,0,0} ealgo ={1,0,0,{0}} algo ={0,1,{0,0}} </pre>
备注	-

IMS_PARM_MEDIA	
Key ID	0x5006
Key 状态	alive
Key 描述	IMS 媒体配置。
结构类型	<pre> typedef struct { uint8_t dtx_enable; uint8_t rsv[3]; } ims_parm_media; </pre>
默认值	<pre> dtx_enable = 1 rsv[3] ={0} </pre>
备注	不支持修改。

IMS_PARM_UE_CAPABILITY	
Key ID	0x5007
Key 状态	alive
Key 描述	IMS UE 能力配置。
结构类型	<pre> typedef struct { uint8_t reserved0; uint8_t reserved1; uint8_t reserved2; uint8_t reserved3; uint8_t reserved4; uint8_t reserved5; uint8_t reserved6; uint8_t reserved7; uint8_t reserved8; uint8_t reserved9; uint8_t reserved10; uint8_t reserved11; uint8_t reserved12; } </pre>

IMS_PARM_UE_CAPABILITY	
	<pre> uint8_t reserved13; uint8_t reserved14; uint8_t reserved15; } uint8_nv_stru; typedef struct { uint16_t reserved0; uint16_t reserved1; uint16_t reserved2; uint16_t reserved3; uint16_t reserved4; uint16_t reserved5; uint16_t reserved6; uint16_t reserved7; } uint16_nv_stru; typedef struct { uint32_t reserved0; uint32_t reserved1; uint32_t reserved2; uint32_t reserved3; uint32_t reserved4; uint32_t reserved5; uint32_t reserved6; uint32_t reserved7; } uint32_nv_stru; typedef struct { uint8_t rsp_sip_timeout; uint8_t image_attr_flag; uint8_t att_feature_flag; uint8_t vdf_switch; uint8_t keep_alive_flag; uint8_t ipsec_flag; uint8_t rsv[2]; uint8_nv_stru uint8_nv_para[16]; uint16_nv_stru uint16_nv_para[16]; uint32_nv_stru uint32_nv_para[8]; } ims_parm_ue_capability; </pre>
默认值	<pre> rsp_sip_timeout =0 image_attr_flag =0 att_feature_flag =0 vdf_switch =0 keep_alive_flag =1 ipsec_flag =0 rsv[2] ={0} uint8_nv_para[16] ={{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 10, 0, 0}, {0}, {0}, {0}, {0}, {0}, {0}, {0}, {0}, {0}, {0}, {0}, {0}, {0}, {0}} uint16_nv_para[16] ={0} uint32_nv_para[8] ={{0, 0, 0, 0, 64, 536870912, 0, 0}, {0}, {0}, {0}, }, {0}, {0}, {0}} </pre>
备注	仅支持 ipsec_flag。

IMS_PARM_SIP_TIMER	
Key ID	0x5008
Key 状态	alive
Key 描述	IMS SIP 定时器配置。
结构类型	<pre>typedef struct { uint32_t timer_t1_value; uint32_t timer_t2_value; uint32_t timer_t4_value; uint32_t timer_ta_value; uint32_t timer_tb_value; uint32_t timer_tc_value; uint32_t timer_td_value; uint32_t timer_te_value; uint32_t timer_tf_value; uint32_t timer_tg_value; uint32_t timer_th_value; uint32_t timer_ti_value; uint32_t timer_tj_value; uint32_t timer_tk_value; } ims_parm_sip_timer;</pre>
默认值	<pre>timer_t1_value =2000 timer_t2_value =16000 timer_t4_value =17000 timer_ta_value =2000 timer_tb_value =128000 timer_tc_value =200000 timer_td_value =200000 timer_te_value =2000 timer_tf_value =128000 timer_tg_value =2000 timer_th_value =128000 timer_ti_value =17000 timer_tj_value =128000 timer_tk_value =128000</pre>
备注	不支持修改。

IMS_NV_EMERGENCY_URN	
Key ID	0x5009
Key 状态	alive
Key 描述	IMS 紧急呼叫 URN。
结构类型	<pre>#define EMC_CATEGOTY_URN_SIZE 6 #define EMC_URN_SIZE 65 typedef struct {</pre>

IMS_NV_EMERGENCY_URN	
	<pre> uint8_t category; uint8_t urn[EMC_URN_SIZE]; uint8_t rsv[2]; } ims_emc_category_urn; typedef struct { ims_emc_category_urn emc_cat_urn[EMC_CATEGOTY_URN_SIZE]; } ims_nv_emergency_urn; </pre>
默认值	{0}
备注	不支持修改。

IMS_PARM_STRING_SWITCH_VALUE	
Key ID	0x500A
Key 状态	alive
Key 描述	IMS 字符串配置。
结构类型	<pre> #define IMS_PARM_STRING_SIZE 129 typedef struct { char phone_context[IMS_PARM_STRING_SIZE]; char phone_context_impu[IMS_PARM_STRING_SIZE]; char dislay_name[IMS_PARM_STRING_SIZE]; uint8_t rsv[1]; } ims_parm_string_switch_value; </pre>
默认值	{0}
备注	暂未使用。

IMSA_NV_IMS_CTRL	
Key ID	0x500B
Key 状态	alive
Key 描述	IMSA 参数配置。
结构类型	<pre> typedef struct { uint8_t single_cause_num; // 临时被 禁原因数量 uint8_t single_cause_list[IMSA_NV_MAX_TEMP_PDN_SINGLE_CAUSE_NUM]; // 临时被禁列表 } imsa_nv_pdn_temp_rej_cause_list; </pre>

IMSA_NV_IMS_CTRL	
	<pre>typedef struct { uint8_t single_cause_num; // 永久被禁原因数量 uint8_t single_cause_list[IMSA_NV_MAX_PERM_PDN_SINGLE_CAUSE_NUM]; // 永久被禁列表 } imsa_nv_pdn_perm_rej_cause_list; typedef struct { uint8_t temp_forb_feature_flag; // 临时被禁特性控制开关 uint8_t perm_forb_feature_flag; // 永久被禁特性控制开关 uint8_t rsv[2]; uint8_t imsa_nv_pdn_temp_rej_cause_list; // PDN 临时被拒原因值列表 uint8_t imsa_nv_pdn_perm_rej_cause_list; // PDN 永久被拒原因值列表 } imsa_pdn_rej_forbidden_ctrl; typedef struct { uint32_t max_time; // RFC5626 计算重试时间时的 maxtime, 单位: s uint32_t base_time; // RFC5626 计算重试时间时的 basetime, 单位: s uint32_t retry_period; // 注册失败后周期性重试时间间隔, 单位: s uint32_t save_retry_times; // 保存 NV 配置的 IMS 注册失败重试次数, 0 表示无限次尝试 imsa_pdn_rej_forbidden_ctrl pdn_rej_forb_ctrl; // 承载被拒控制模块 } imsa_nv_ims_ctrl;</pre>
默认值	<pre>max_time =1800 base_time =30 retry_period =0 save_retry_times =5</pre>
备注	-

IMSA_NV_IMS_CONFIG	
Key ID	0x500C
Key 状态	alive
Key 描述	IMSA 参数配置。
结构类型	<pre>typedef struct { uint8_t user_info_flag;</pre>

IMSA_NV_IMS_CONFIG	
	<pre>uint8_t auth_type; uint8_t rsv[2]; char impi[128]; char impu[128]; char domain[128]; char pass_word[128]; } imsa_nv_ims_config;</pre>
默认值	{0}
备注	暂未使用。

IMSA_UE_CAP	
Key ID	0x500D
Key 状态	alive
Key 描述	IMSA UE 能力配置。
结构类型	<pre>typedef struct { uint8_t voice_call_on_ims_support_flag; // IMS 语音使能项, 1: 支持, 0: 不支持 uint8_t sms_on_ims_support_flag; // IMS 短信使能项, 1: 支持, 0: 不支持 uint16_t call_rsrp_threshold; } imsa_ue_cap;</pre>
默认值	<pre>voice_call_on_ims_support_flag =1 sms_on_ims_support_flag =1 call_rsrp_threshold =-110</pre>
备注	-

IMS_PCSCF_DISCOVERY_POLICY	
Key ID	0x500E
Key 状态	alive
Key 描述	IMSA NV 配置 PCSCF 地址。
结构类型	<pre>#define IMSA_NV_IPV4_ADDR_LEN 4 #define IMSA_NV_IPV6_ADDR_LEN 16 typedef struct { uint32_t bit_op_prim_pcscf_addr : 1; uint32_t bit_op_sec_pcscf_addr : 1; // 如</pre>

IMS_PCSCF_DISCOVERY_POLICY	
	果 bitOpSecPcscfAddr 为 1, bitOpPrimPcscfAddr 也必须为 1 <pre> uint32_t bit_op_spare : 30; uint8_t prim_pcscf_addr[IMSA_NV_IPV4_ADDR_LEN]; uint8_t sec_pcscf_addr[IMSA_NV_IPV4_ADDR_LEN]; } imsa_nv_pdp_ipv4_pcscf; typedef struct { uint32_t bit_op_prim_pcscf_addr : 1; uint32_t bit_op_sec_pcscf_addr : 1; // 如 果 bitOpSecPcscfAddr 为 1, bitOpPrimPcscfAddr 也必须为 1 uint32_t bit_op_spare : 30; uint8_t prim_pcscf_addr[IMSA_NV_IPV6_ADDR_LEN]; uint8_t sec_pcscf_addr[IMSA_NV_IPV6_ADDR_LEN]; } imsa_nv_pdp_ipv6_pcscf; typedef struct { imsa_pcscf_discovery_policy pcscf_discover_policy; // P-CSCF 获取策略 uint8_t rsv[3]; imsa_nv_pdp_ipv4_pcscf ipv4_pcscf; // 承载 IPv4 P-CSCF 信息 imsa_nv_pdp_ipv6_pcscf ipv6_pcscf; // 承载 IPv6 P-CSCF 信息 } imsa_nv_pcscf_discovery_policy; </pre>
默认值	<pre> pcscf_discover_policy =IMSA_PCSCF_DISCOVERY_POLICY_PCO rsv[3] ={0} ipv4_pcscf ={0} ipv6_pcscf ={0} </pre>
备注	不支持修改。

IMS_SIP_PORT	
Key ID	0x500F
Key 状态	alive
Key 描述	IMS 注册端口范围。
结构类型	<pre> typedef struct { uint16_t ims_min_port; uint16_t ims_max_port; } ims_sip_port; </pre>

IMS_SIP_PORT	
默认值	ims_min_port =5000 ims_max_port =32000
备注	不支持修改。

IMS_COMM_PARA_CONFIG	
Key ID	0x5010
Key 状态	alive
Key 描述	IMS 语音端口号范围。
结构类型	<pre>typedef struct { uint32_t speech_start; uint32_t speech_end; uint32_t video_start; uint32_t video_end; } ims_comm_para_config;</pre>
默认值	{0}
备注	暂未使用。

IMSA_NV_CALL_IMS_CCWA_CTRL_MODE	
Key ID	0x5011
Key 状态	alive
Key 描述	-
结构类型	<pre>typedef enum { IMSA_CALL_CCWA_CTRL_BY_NW = 0, IMSA_CALL_CCWA_CTRL_BY_UE = 1, IMSA_CALL_CCWA_CTRL_MODE_BUTT = 0xFF, } imsa_call_ccwa_ctrl_mode; typedef enum { IMSA_CALL_CCWAI_MODE_DISABLE = 0, IMSA_CALL_CCWAI_MODE_ENABLE = 1, IMSA_CALL_CCWAI_MODE_BUTT = 0xFF, } imsa_call_ccwai_mode; typedef struct { imsa_call_ccwa_ctrl_mode ccwa_ctrl_mode; imsa_call_ccwai_mode ccwai_mode; uint8_t rsv[2]; } imsa_nv_call_ims_ccwa_ctrl_mode;</pre>

IMSA_NV_CALL_IMS_CCWA_CTRL_MODE	
默认值	ccwa_ctrl_mode =IMSA_CALL_CCWA_CTRL_BY_UE ccwai_mode =IMSA_CALL_CCWAI_MODE_ENABLE rsv[2] = {0}
备注	不支持修改。